

NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



INTRODUCTION TO DATABASE

FINAL PROJECT REPORT

Sports Ticketing Management System

Supervisor: Dr. Tran Hung

Student: Ta Van Sang

Student ID: 11247221

Class: DS66B

Hanoi, 2026

Contents

Preface: Original Assignment	3
1 Project Overview	7
1.1 Project Objective	7
1.2 System Scope	7
1.3 Technology Stack	7
2 System Analysis and Design	8
2.1 Requirements Analysis	8
2.1.1 Functional Requirements	8
2.1.2 Non-Functional Requirements	8
2.2 Data Model Design (ER Diagram)	8
2.2.1 Entities and Attributes	8
2.2.2 Relationships Between Entities	8
2.2.3 ERD (Entity-Relationship Diagram)	8
3 Database Design and Implementation	10
3.1 Table Structures (Relational Schema)	10
3.1.1 Table: Events	10
3.1.2 Table: Tickets	10
3.1.3 Table: Customers	10
3.1.4 Table: BoxOffices	10
3.1.5 Table: Seats	10
4 Advanced Database Objects	13
4.1 Indexes	13
4.2 Views	13
4.3 Stored Procedures	13
4.4 User-Defined Functions	13
4.5 Triggers	14
5 Database Security and Administration	15
5.1 Role-Based User Access Control	15
5.2 Customer Data Encryption	15
5.3 Backup and Recovery	15
5.4 Query Performance Optimization	15
6 Python Application Development	16
6.1 Database Connection	16
6.2 Dashboard	16
6.3 Events Page	16
6.4 Book Ticket	16
6.5 Customers Page	17
6.6 Revenue Page	17
6.7 Running the Application	18



7	Conclusion and Recommendations	21
7.1	Project Outcomes	21
7.2	Limitations	21
7.3	Recommendations for Future Development	21



Preface: Original Assignment

This report is submitted as the final project for the course **Introduction to Database**, under the supervision of **Dr. Tran Hung**, Faculty of Data Science and Artificial Intelligence, National Economics University.

The project is based on the official assignment specification issued by DATCOM Lab (Project 16: Sports Ticketing Management System). The original assignment document is attached below for reference.



PROJECT 16: SPORTS TICKETING MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to build a system to manage the sale of tickets for sports events, including scheduling, seat booking, customer management, and revenue reporting, ensuring fast, convenient, and accurate service for both organizers and spectators.

Database and Programming Languages

- Database Management System: MySQL
- Programming Language: Python

Detailed Project Requirements

System Analysis and Requirements

- Manage data related to sporting events, ticket types, customers, seating arrangements, ticket sales, and stadium box offices.

Main Functionalities

- Event scheduling and match information management.
- Ticket type and pricing management.
- Customer management and ticket purchasing history.
- Seat booking and ticket availability tracking.
- Ticket sales and box office management.
- Reporting on revenue, ticket sales, and event popularity.



Database Design and Implementation

1. Data Model Design

- Create an ER diagram showing the relationship among events, customers, tickets, and seating.
- Convert ER diagram to relational schema with PKs, FKs, and constraints.

2. Table Structures

- Events (EventID, EventName, EventDate, Venue)
- Tickets (TicketID, EventID, TicketType, Price)
- Customers (CustomerID, CustomerName, PhoneNumber, Address)
- BoxOffices (BoxOfficeID, OfficeName, Address)
- Seats (SeatID, EventID, SeatNumber, SeatType)

3. Sample Data

- Insert 510 rows for each table.
- Use MySQL Workbench to generate a schema diagram.

Advanced Database Objects

- **Indexes:** Speed up queries on available seats and customer purchases.
- **Views:** Generate views for sold-out events, revenue by event, or seat availability.
- **Stored Procedures:** Automate ticket booking and generate revenue summaries.
- **User Defined Functions:** Calculate total revenue per event or total tickets sold.
- **Triggers:** Update seat status when a ticket is sold or canceled.

Database Security and Administration

- Define user roles such as cashier, admin, and manager with appropriate access rights.
- Encrypt customer information and apply secure transaction handling.
- Set up backup plans and disaster recovery procedures.
- Optimize query performance for high-traffic sales periods.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Booking System:** Implement Python scripts for searching events, booking tickets, and canceling orders.
- **Reporting Module:** Generate real-time sales statistics and revenue dashboards.
- **User Interface:** Provide a console-based or optional GUI for box office operations.



Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize project success and performance.
- Recommend future enhancements such as QR code integration, online payment, or mobile ticketing apps.

References

- List all technical documents, libraries, and data sources used in the project.



1 Project Overview

1.1 Project Objective

The Sports Ticketing Management System is designed to digitize and automate the full ticket management lifecycle for sporting events. Rather than managing sales manually through paper records or disconnected spreadsheets, this system centralizes all operations into a single relational database, ensuring data consistency and enabling real-time reporting.[3]

The system solves several real-world problems common to large-scale sporting events:

- Duplicate seat bookings due to lack of centralized tracking.
- Difficulty auditing revenue across multiple box offices.
- Inability to efficiently retrieve a customer's purchase history.
- Lack of role-based access, leading to data integrity risks.

1.2 System Scope

The system covers the following core functional areas:

- Sports event management (schedules, venues, match information).
- Ticket type and pricing management.
- Customer management (personal information, transaction history).
- Seat management (booking, status tracking of available/sold seats).
- Box office and staff management.
- Statistical reporting on revenue and ticket sales per event.

1.3 Technology Stack

Table 1: Technology Stack

Component	Technology	Purpose
Database Management System	MySQL 8.0+	Data storage and management
Programming Language	Python 3.x	Application logic and reporting
DB Connector	mysql-connector-python	Python ↔ MySQL communication
DB Design Tool	MySQL Workbench	ERD design and schema visualization
Version Control	Git / GitHub	Source code management

2 System Analysis and Design

2.1 Requirements Analysis

2.1.1 Functional Requirements

Based on the project specification, the system must satisfy the following functional requirements:[2]

- **Event Management:** Add, edit, and delete sporting event records; list events by date and venue.
- **Ticket Management:** Create ticket types (VIP, Standard, Student) with pricing per event; check remaining availability.
- **Customer Management:** Register new customers; view purchase history per customer.
- **Seat Booking and Sales:** Select a specific seat; complete a ticket sale transaction; cancel bookings when necessary.
- **Reporting:** Revenue reports per event; ticket sales rate; list of sold-out events.
- **System Administration:** Role-based user permissions (admin, cashier, manager); database backup and recovery.

2.1.2 Non-Functional Requirements

- **Performance:** Query response time under 2 seconds under normal load conditions.
- **Security:** Encryption of sensitive customer data; strict access control enforcement.
- **Data Integrity:** No duplicate seat bookings, enforced by transaction management and triggers.
- **Scalability:** Flexible database structure that accommodates future feature additions.

2.2 Data Model Design (ER Diagram)

2.2.1 Entities and Attributes

The system consists of 5 main entities:

2.2.2 Relationships Between Entities

All relationships in this system are 1:N (one-to-many). The Tickets table acts as the central junction, holding foreign keys to Events, Customers, Seats, and BoxOffices. The Customers–Tickets relationship is 1:N rather than N:M because each Ticket row represents a single physical seat assigned to exactly one customer.

2.2.3 ERD (Entity-Relationship Diagram)

Table 2: System Entities and Attributes

Entity	Key Attributes	Description
Events	EventID (PK), EventName, EventDate, Venue	Stores information about each sporting event
Tickets	TicketID (PK), EventID (FK), TicketType, Price	Ticket types and prices linked to events
Customers	CustomerID (PK), CustomerName, PhoneNumber, Address	Customer registration and profile data
BoxOffices	BoxOfficeID (PK), OfficeName, Address	Physical or virtual ticket sales counters
Seats	SeatID (PK), EventID (FK), SeatNumber, SeatType	Seat information per event

Table 3: Entity Relationships

Relationship	Cardinality	Description
Events – Tickets	1:N	One event has many ticket records
Events – Seats	1:N	One event has many seats
Customers – Tickets	1:N	One customer buys many tickets; each ticket belongs to one customer
BoxOffices – Tickets	1:N	One box office processes many tickets

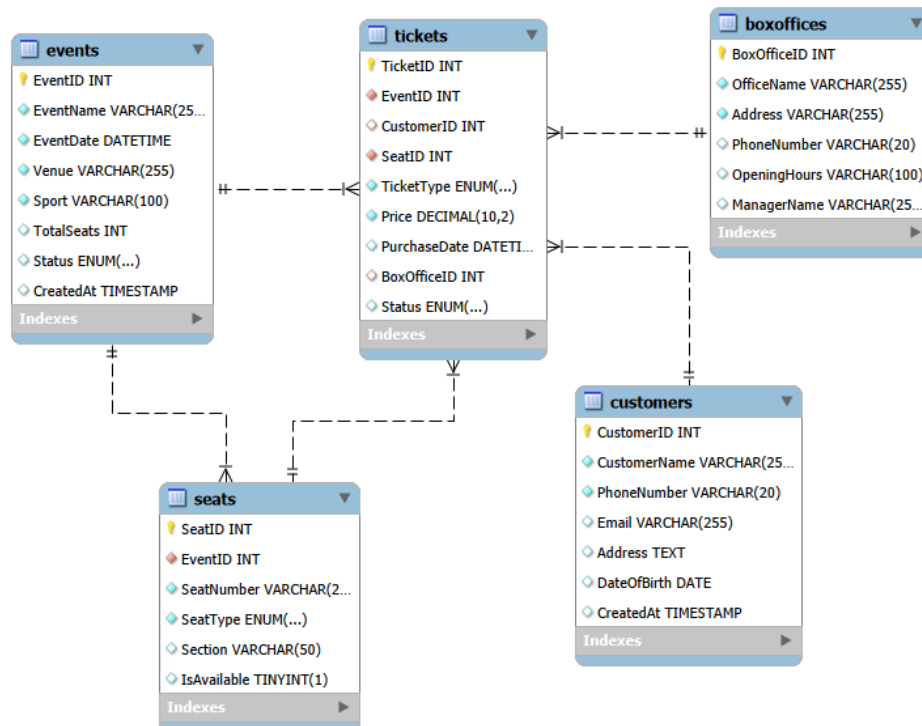


Figure 1: ERD of the Sports Ticketing Management System

3 Database Design and Implementation

3.1 Table Structures (Relational Schema)

3.1.1 Table: Events

Stores information about each sporting event. The **Status** column uses **ENUM** to restrict values to a controlled vocabulary, preventing invalid states from being inserted.

Table 4: Table: Events

Column Name	Data Type	Constraints	Description
EventID	INT	PK, AUTO_INCREMENT	Unique event identifier
EventName	VARCHAR(255)	NOT NULL	Name of the event
EventDate	DATETIME	NOT NULL	Date and time of the event
Venue	VARCHAR(255)	NOT NULL	Location/venue name
Sport	VARCHAR(100)	NOT NULL	Type of sport
TotalSeats	INT	DEFAULT 0	Total seating capacity
Status	ENUM	'Upcoming', 'Ongoing', 'Completed', 'Cancelled'	Current event status
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

3.1.2 Table: Tickets

Each row represents one physical ticket for a specific seat. **CustomerID** is nullable because a ticket may exist without a buyer (**Status** = 'Available'). Once sold, **CustomerID** is populated and **Status** changes to 'Sold' – this transition is enforced atomically inside a stored procedure to prevent race conditions.

3.1.3 Table: Customers

PhoneNumber and **Email** are both marked **UNIQUE** to prevent duplicate customer registrations. Email is additionally subject to AES-256 encryption at rest (see Section 5.2).

3.1.4 Table: BoxOffices

3.1.5 Table: Seats

The **IsAvailable** flag prevents double-booking. It is automatically updated by a trigger whenever a ticket's **Status** changes, ensuring the **Seats** table always reflects true availability without requiring the application to synchronize it manually.

The full SQL scripts for database creation and sample data insertion are available on GitHub:[2]
<https://github.com/tvsng23/sports-ticketing-management>



Table 5: Table: Tickets

Column Name	Data Type	Constraints	Description
TicketID	INT	PK, AUTO_INCREMENT	Unique ticket identifier
EventID	INT	FK → Events, NOT NULL	Associated event
CustomerID	INT	FK → Customers, NULL	Buyer (NULL = unsold)
SeatID	INT	FK → Seats, NOT NULL	Associated seat
TicketType	ENUM	VIP / Standard / Student / Group	Ticket category
Price	DECIMAL(10,2)	NOT NULL, CHECK (Price ≥ 0)	Ticket price
PurchaseDate	DATETIME	NULL	Date and time of purchase
BoxOfficeID	INT	FK → BoxOffices, NULL	Issuing box office
Status	ENUM	Available / Sold / Cancelled	Ticket status

Table 6: Table: Customers

Column Name	Data Type	Constraints	Description
CustomerID	INT	PK, AUTO_INCREMENT	Unique customer identifier
CustomerName	VARCHAR(255)	NOT NULL	Full name
PhoneNumber	VARCHAR(20)	UNIQUE, NOT NULL	Phone number
Email	VARCHAR(255)	UNIQUE	Email address (encrypted at rest)
Address	TEXT	NULL	Mailing address
DateOfBirth	DATE	NULL	Date of birth
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp

Table 7: Table: BoxOffices

Column Name	Data Type	Constraints	Description
BoxOfficeID	INT	PK, AUTO_INCREMENT	Unique box office identifier
OfficeName	VARCHAR(255)	NOT NULL	Name of the sales counter
Address	VARCHAR(255)	NOT NULL	Physical address
PhoneNumber	VARCHAR(20)	NULL	Contact phone
OpeningHours	VARCHAR(100)	NULL	Operating hours
ManagerName	VARCHAR(255)	NULL	Name of the manager in charge



Table 8: Table: Seats

Column Name	Data Type	Constraints	Description
SeatID	INT	PK, AUTO_INCREMENT	Unique seat identifier
EventID	INT	FK → Events, NOT NULL	Associated event
SeatNumber	VARCHAR(20)	NOT NULL	Seat label (e.g. A01, B12)
SeatType	ENUM	VIP / Standard / Student	Seat category
Section	VARCHAR(50)	NULL	Stadium section (Block A, B, C...)
IsAvailable	TINYINT(1)	DEFAULT 1	1 = Available, 0 = Occupied



4 Advanced Database Objects

4.1 Indexes

Without indexes, MySQL must perform a full table scan on every query – reading every row to find matching records. For a high-traffic ticketing system, this would be unacceptably slow during peak sales periods. Four indexes were created to accelerate the most frequent query patterns:

Table 9: Database Indexes

Index Name	Table	Columns	Purpose
idx_seats_available	Seats	EventID, IsAvailable	Find all empty seats for a given event
idx_tickets_customer	Tickets	CustomerID, Status	Retrieve all tickets by customer
idx_tickets_event	Tickets	EventID, Status	Revenue reporting per event
idx_events_date	Events	EventDate	Filter upcoming events by date

A composite index on (**EventID**, **IsAvailable**) is more efficient than two separate indexes because both conditions always appear together in the **WHERE** clause of the seat availability query.

4.2 Views

Three views were created to simplify complex reporting queries and enforce consistent business logic:[5]

vw_revenue_by_event aggregates total revenue and tickets sold per event. It uses **LEFT JOIN** so that events with zero sales still appear in the report, and **CASE WHEN** inside **SUM()** ensures only 'Sold' tickets contribute to revenue.

vw_soldout_events identifies events where every available ticket has been sold, by comparing total ticket count against sold ticket count using **HAVING**.

vw_seat_availability provides a breakdown of available versus occupied seats by seat type for each event, supporting real-time capacity planning.

4.3 Stored Procedures

Two stored procedures handle the core transactional logic of the system.[6]

sp_book_ticket encapsulates the entire ticket booking flow inside a single database transaction, guaranteeing atomicity. It accepts **CustomerID**, **SeatID**, and **BoxOfficeID** as inputs and returns a **TicketID** and status message as outputs.

The key design decision is **SELECT ... FOR UPDATE** on the **Seats** row before modifying it. This locks the row for the duration of the transaction, preventing two concurrent cashiers from booking the same seat simultaneously – a race condition known as a *Lost Update*[4]. If the seat is already taken (**IsAvailable** = 0), the procedure rolls back immediately and returns an error message.

sp_revenue_report accepts an **EventID** and returns a breakdown of revenue grouped by ticket type, which powers the reporting module in the Python application.

4.4 User-Defined Functions

Two scalar functions encapsulate frequently-used calculations:



- `fn_total_revenue(EventID)` – returns the sum of prices for all sold tickets for a given event.
- `fn_tickets_sold(EventID)` – returns the count of sold tickets for a given event.

Both are marked `READS SQL DATA` and `DETERMINISTIC`, allowing MySQL to cache results when the same `EventID` is queried multiple times within a report. The full implementation is available in `sql/06_functions.sql` on GitHub.

4.5 Triggers

Two `AFTER UPDATE` triggers automate the synchronization of seat availability whenever a ticket's status changes:

`trg_ticket_sold` fires when a ticket transitions from 'Available' to 'Sold', setting the corresponding seat's `IsAvailable` to 0.

`trg_ticket_cancelled` fires when a ticket transitions from 'Sold' to 'Cancelled', resetting `IsAvailable` to 1 so the seat becomes bookable again.

The `OLD` and `NEW` keywords within the trigger body allow access to column values before and after the update, enabling precise conditional logic.

5 Database Security and Administration

5.1 Role-Based User Access Control

Three user roles are defined following the principle of least privilege – each role is granted only the permissions required to perform its specific job function, minimizing the risk of accidental or malicious data modification.[4]

Table 10: User Roles and Privileges

Role	Privileges Granted	Purpose
admin_ticketing	ALL PRIVILEGES on sports_ticketing.*	Full administrative access for database maintenance and schema changes
manager_ticketing	SELECT on all tables and views; EXECUTE sp_revenue_report	Read-only access to reports and statistics; cannot modify data
cashier_ticketing	SELECT on Events/Seats/Tickets; INSERT/UPDATE on Customers; EXECUTE sp_book_ticket	Can sell tickets and register customers; cannot view reports or modify events

5.2 Customer Data Encryption

Customer email addresses are encrypted at rest using MySQL's `AES_ENCRYPT` function with a 256-bit key before being stored. The encrypted bytes are converted to hexadecimal using `HEX()` for safe storage in a `VARCHAR` column. To retrieve the original value, `AES_DECRYPT` and `UNHEX` are applied. This protects customer data in the event of unauthorized database access – an attacker who obtains raw table data cannot read emails without the encryption key. Full scripts are available in `sql/08_security.sql` on GitHub.

5.3 Backup and Recovery

Database backups are performed via MySQL Workbench's Data Export feature (*Server* → *Data Export*), which produces a self-contained SQL dump file. This file can be used to fully restore the database using MySQL Workbench's Data Import feature or the `mysql` command-line client. A backup should be taken before any schema changes or bulk data operations.

5.4 Query Performance Optimization

The `EXPLAIN` statement was used to analyze the execution plan of complex multi-table `JOIN` queries. Key indicators are the `type` column (`ref` or `eq_ref` is good; `ALL` indicates a full table scan and should be avoided) and the `key` column (`NULL` means no index is being used). After creating the indexes in Section 4.1, re-running `EXPLAIN` confirmed that MySQL switched from full table scans to index-range scans on the `Tickets` and `Seats` tables.

6 Python Application Development

A Flask web application was developed to demonstrate that all database objects function correctly through a real client application. The application uses `mysql-connector-python` to communicate with MySQL and is structured into three modules: connection management, booking, and reporting.

6.1 Database Connection

The `get_connection()` function centralizes all connection parameters and wraps `mysql.connector.connect()` in a `try/except` block. Every function that needs database access calls `get_connection()` and explicitly closes both the cursor and connection in a `finally` block, regardless of whether an error occurred. This prevents connection leaks, which can exhaust MySQL's connection pool under load.

```
1 import mysql.connector
2
3 def get_connection():
4     try:
5         conn = mysql.connector.connect(
6             host="localhost",
7             user="cashier_ticketing",
8             password="your_password",
9             database="sports_ticketing"
10        )
11        return conn
12    except mysql.connector.Error as e:
13        print(f"Connection error: {e}")
14        raise
```

Listing 1: Database connection function

6.2 Dashboard

The Dashboard page aggregates four key metrics – total events, registered customers, tickets sold, and total revenue – using direct `COUNT` and `SUM` queries. It also displays a Revenue by Event bar chart built from a `GROUP BY` query joining Events and Tickets, and a Recent Transactions table showing the eight most recent sales.

6.3 Events Page

The Events page displays all events enriched with computed columns: total seats created, available seats, tickets sold, and revenue per event. These are calculated via a single `GROUP BY` query joining Events, Seats, and Tickets with conditional aggregation. Each event row includes a direct **Book** button that pre-selects that event on the booking page.

6.4 Book Ticket

The booking page demonstrates the full transactional flow. The user first selects an event from a dropdown, which triggers a `GET` request to reload the page with available seats fetched via a `JOIN` between Seats and Tickets filtering on `IsAvailable = 1` and `Status = 'Available'`. On `POST`, the application calls `sp_book_ticket` using three separate `execute()` calls – setting the `@tid` and `@msg` session variables first, then calling the procedure, then reading the output

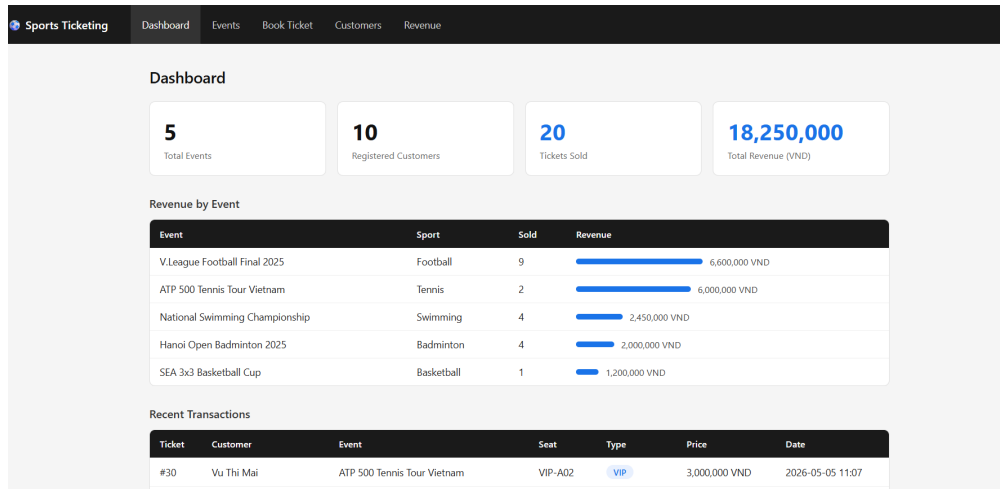


Figure 2: Flask web application — Dashboard

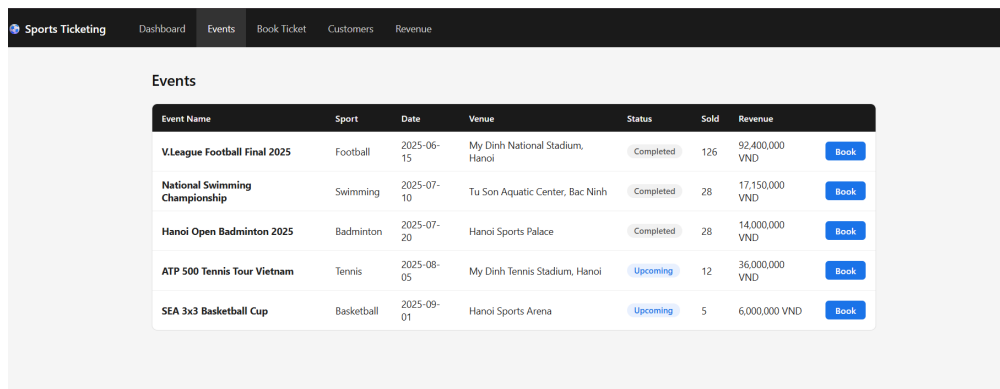


Figure 3: Events page with status badges and revenue per event

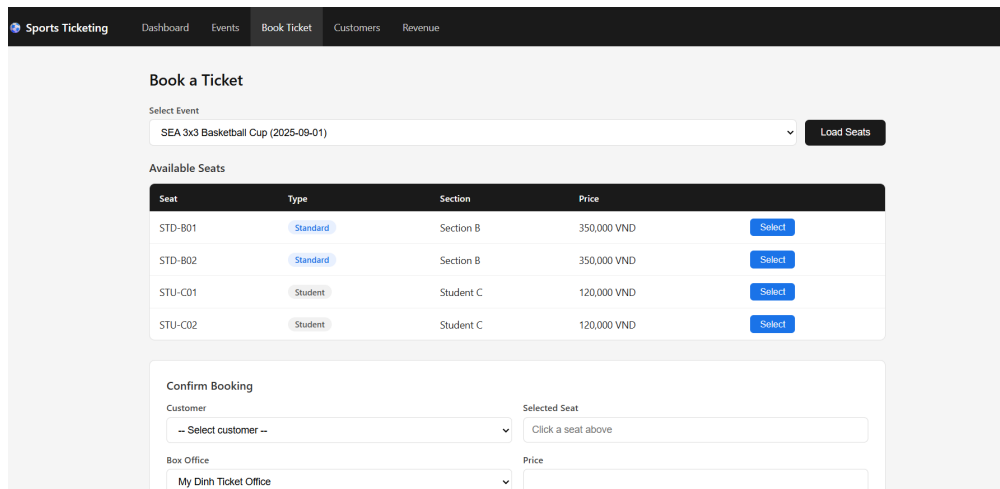
variables back with `SELECT`. This approach is required because `mysql-connector-python` does not support multiple statements in a single `execute()` call.

6.5 Customers Page

The Customers page lists all registered customers ranked by total amount spent, computed via a `LEFT JOIN` between Customers and Tickets filtered on `Status = 'Sold'`. This demonstrates that purchase history is correctly tracked through the `CustomerID` foreign key in the Tickets table.

6.6 Revenue Page

The Revenue page pulls directly from the two views created in Section 4.2. The upper section queries `vw_revenue_by_event` and renders each event's total revenue as an inline progress bar scaled relative to the highest-revenue event. The lower section queries `vw_seat_availability` and displays occupancy rates per seat type per event, with the bar turning red when occupancy



Seat	Type	Section	Price	Select
STD-B01	Standard	Section B	350,000 VND	Select
STD-B02	Standard	Section B	350,000 VND	Select
STU-C01	Student	Student C	120,000 VND	Select
STU-C02	Student	Student C	120,000 VND	Select

Figure 4: Book Ticket page showing available seats for selection

exceeds 70%. This demonstrates that views simplify complex reporting to a single **SELECT** * statement in the application layer.

6.7 Running the Application

The application requires two dependencies:

```
1 pip install flask mysql-connector-python
```

Listing 2: Install dependencies

After updating the database credentials in `app.py`, start the server with:

```
1 python app.py
```

Listing 3: Start the Flask server

The application is then accessible at <http://127.0.0.1:5000>. Full source code is available at: <https://github.com/tvsng23/sports-ticketing-management>



Sports Ticketing | Dashboard | Events | **Book Ticket** | Customers | Revenue

✓ Booking successful! Ticket ID = 36

✓ Booking successful! Ticket ID = 36

Book a Ticket

Select Event
SEA 3x3 Basketball Cup (2025-09-01) Load Seats

Available Seats

Seat	Type	Section	Price	
STD-B02	Standard	Section B	350,000 VND	Select
STU-CD1	Student	Student C	120,000 VND	Select
STU-CD2	Student	Student C	120,000 VND	Select

Confirm Booking

Customer: -- Select customer -- | Selected Seat: Click a seat above

Figure 5: Successful booking confirmation

Sports Ticketing | Dashboard | Events | **Book Ticket** | **Customers** | Revenue

Customers

Name	Phone	Address	Tickets Bought	Total Spent
Nguyen Thanh Long	0901234567	12 Tran Duy Hung, Hanoi	3	5,800,000 VND
Vu Thi Mai	0934567890	22 Kim Ma, Hanoi	3	4,300,000 VND
Pham Thu Huong	0912345678	45 Le Van Luong, Hanoi	1	2,000,000 VND
Do Minh Khoa	0923456789	89 Nguyen Chi Thanh, Hanoi	2	2,000,000 VND
Le Thi Lan	0967890123	88 Nguyen Trai, Hanoi	3	1,800,000 VND
Tran Quoc Bao	0956789012	15 Doi Can, Hanoi	2	800,000 VND
Nguyen Bich Ngoc	0989012345	55 Lang Ha, Hanoi	2	600,000 VND
Hoang Duc Nam	0945678901	67 Xuan Thuy, Hanoi	1	500,000 VND
Phan Van Duc	0978901234	33 Hoang Cau, Hanoi	2	300,000 VND
Cao Minh Tri	0990123456	77 Cau Giay, Hanoi	1	150,000 VND

Figure 6: Customers page showing purchase history and total spend

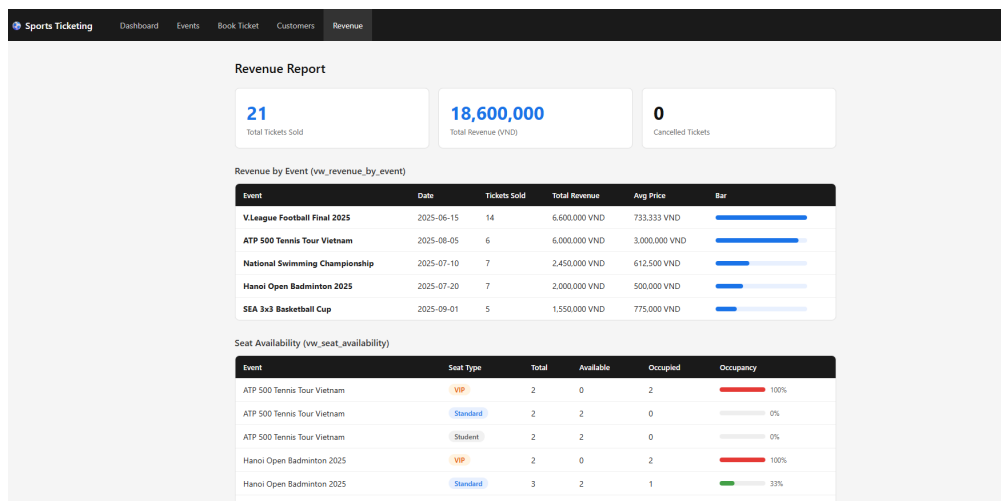


Figure 7: Revenue page powered by vw_revenue_by_event and vw_seat_availability

7 Conclusion and Recommendations

7.1 Project Outcomes

This project successfully designed and implemented a fully functional Sports Ticketing Management System, demonstrating how a well-structured relational database enforces business rules at the data layer rather than relying solely on application-level logic.

Database Design and Implementation. A five-table relational database was designed in Third Normal Form with full integrity constraints including primary keys, foreign keys, `CHECK`, `UNIQUE`, `DEFAULT`, and `ENUM`. The schema was validated through MySQL Workbench's Reverse Engineer function, confirming that all relationships and constraints were correctly implemented.[3, 1]

Advanced Database Objects. All four categories of advanced objects were implemented: indexes to accelerate the two most frequent query patterns; views to encapsulate revenue and seat availability reporting logic; stored procedures to handle transactional ticket booking with row-level locking; user-defined functions for inline revenue calculation; and triggers for automatic seat status synchronization.

Security and Administration. Role-based access control was implemented across three user roles following the principle of least privilege. Customer email addresses are protected with AES-256 encryption at rest.

Python Application. A Flask web application was developed providing five functional pages that exercise all database objects through a real client interface. The booking flow demonstrates the ACID properties of the transaction in `sp_book_ticket`. [4]

7.2 Limitations

Three limitations of the current system are worth acknowledging:

1. The web interface runs locally and is not deployed to a public server, meaning it cannot be accessed outside the development machine without additional infrastructure.
2. There is no online payment integration – ticket purchases are assumed to be cash or card transactions handled at a physical box office.
3. The system has no authentication layer for the web application itself. In a production deployment, login and session management would be required to enforce the role-based permissions defined in Section 5.

7.3 Recommendations for Future Development

QR Code Ticketing. Each sold ticket could generate a unique QR code stored as a URL in the Tickets table. Scanning the QR code at the stadium gate would trigger a validation query against the database, replacing paper tickets entirely.

Online Payment Integration. Connecting to Vietnamese payment gateways such as VNPay or MoMo would allow customers to purchase tickets remotely. The Tickets table schema already supports this extension – a payment reference column could be added without restructuring existing tables.

Mobile Application. A React Native or Flutter mobile app consuming a REST API layer above the existing database would allow customers to browse events, select seats, and purchase tickets from their phones.



Dynamic Pricing. A machine learning model trained on historical sales data could adjust ticket prices based on demand, time remaining before the event, and seat type.

Cloud Deployment. Migrating the MySQL instance to AWS RDS or Google Cloud SQL, and deploying the Flask application on a cloud server, would enable horizontal scalability to handle high-traffic sales periods.



References

- [1] Carlos Coronel and Steven Morris. *Database Systems: Design, Implementation, and Management*. 14th. Cengage Learning, 2022.
- [2] DATCOM Lab, NEU. *Project 16: Sports Ticketing Management System — Project Specification*. National Economics University. 2025.
- [3] Raghu Ramakrishnan and Johannes Gehrke. *Database Management Systems*. 3rd. McGraw-Hill, 2003.
- [4] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 7th. McGraw-Hill, 2019.
- [5] Hung Tran. *Lecture 2: View*. DATCOM Lab, College of Technology, National Economics University. 2025.
- [6] Hung Tran and Duc Manh Tran. *Lecture 4: Stored Procedures*. DATCOM Lab, College of Technology, National Economics University. 2025.